FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA — FIAP
Graduação em Análise e Desenvolvimento de Sistemas

Equipe: Urbanlink

Integrantes:
Felipe Falcão Ferreira
Felipe Fernandes Cabral Mota
Rafael Andres da Silva
Vitor Reis Souza
Vitor Rodrigues Ferreira

URBANLINK

Sistema de Conversão de Pontos em Créditos de Transporte Público
Disciplina: Domain Driven Design Using Java — Sprint 4

São Paulo
2026

SUMÁRIO

1 OBJETIVO E ESCOPO DO PROJETO....3
1.1 Propósito do Sistema....3
1.2 Escopo e Limites de Atuação....3
1.3 Regras de Negócio Principais....4
2 DESCRIÇÃO DAS FUNCIONALIDADES....4
2.1 Principais Funcionalidades....4
2.2 Arquitetura em Camadas....5
2.3 Endpoints da API....5
2.4 Tratamento de Erros e Códigos de Status....6
3 PROTÓTIPO DO SISTEMA....8
3.1 Descrição das Telas....9
4 MODELO DE ENTIDADE-RELACIONAMENTO (MER)....10
4.1 Entidades e Relacionamentos....10
5 DIAGRAMA DE CLASSES....11
5.1 Visão Geral das Classes....11
5.2 Representação UML....11
5.3 Descrição dos Relacionamentos....14
6 PROCEDIMENTOS PARA RODAR A APLICAÇÃO....14
6.1 Ferramentas Utilizadas....14
6.2 Versões Necessárias....14
6.3 Como Importar o Projeto....15
6.4 Variáveis de Ambiente e Banco de Dados....15
6.5 Passo a Passo de Execução....15
6.6 Como Testar a API....16
6.7 Publicação (Deploy)....16
7 CONSIDERAÇÕES FINAIS....16

1 OBJETIVO E ESCOPO DO PROJETO

1.1 Propósito do Sistema

O Urbanlink é um módulo da plataforma SoulUp responsável por conectar o ambiente digital de acúmulo de pontos sustentáveis à utilização de transporte público. O propósito central do sistema é permitir que usuários convertam pontos acumulados por meio de ações com impacto ambiental positivo realizadas na plataforma em créditos reais de transporte, financiando total ou parcialmente o custo das passagens.

A solução promove um ciclo virtuoso: ao incentivar comportamentos sustentáveis e, ao mesmo tempo, subsidiar o uso do transporte público, o sistema reduz diretamente a emissão de gases de efeito estufa, contribuindo para as metas de descarbonização urbana. Cada viagem realizada via Urbanlink é quantificada em termos de CO2 evitado, gerando um relatório de impacto individual e consolidado para a plataforma.

Nesta etapa (Sprint 4) o sistema deixou de ser uma aplicação de console e passou a ser uma API REST desenvolvida com Spring Boot, organizada em camadas (controller, service, domínio e infraestrutura) e integrada ao banco de dados Oracle, de modo que qualquer interface (como o front-end web apresentado na seção 3 ou outra aplicação) possa consumir os serviços.

1.2 Escopo e Limites de Atuação

O sistema atua dentro dos seguintes limites funcionais:

- **Conversão de pontos em vouchers:** núcleo do sistema em que o usuário informa o percentual de cobertura desejado (0-100%) e o sistema deduz os pontos correspondentes, emitindo um voucher com código único.
- **Prevenção de fraudes:** o serviço implementa validação antifraude que verifica a titularidade do voucher, limita a quantidade de vouchers emitidos por usuário no mesmo dia e bloqueia a utilização de vouchers que não estejam ativos.
- **Medição de impacto ambiental:** a cada voucher utilizado, o sistema registra o CO2 evitado (por modal de transporte) e calcula a equivalência em árvores absorventes, gerando um relatório por usuário.
- **Perfis de usuário:** o sistema distingue usuários comuns de usuários Premium, que recebem multiplicador de bônus na conversão e possuem limite mensal de resgates configurável.
- **Cadastros (CRUD):** usuários, transportes (públicos e integrados), vouchers e

impactos ambientais são expostos por endpoints REST com os verbos GET, POST, PUT e DELETE.

1.3 Regras de Negócio Principais

Quadro 1 — Regras de negócio do sistema

Regra	Descrição
Taxa de conversão	10 pontos = R$ 1,00
Cobertura parcial	O usuário pode cobrir entre 0% e 100% do valor da passagem com pontos
Bônus Premium	Usuários Premium têm multiplicador de bônus aplicado na conversão (pagam menos pontos pelo mesmo crédito)
Limite mensal	Usuários Premium possuem limite mensal de resgates configurável
Antifraude — titularidade	O voucher só pode ser utilizado pelo usuário que o gerou
Antifraude — limite diário	Cada usuário pode gerar no máximo 5 vouchers por dia; acima disso a conversão é recusada
Status do voucher	Vouchers seguem o fluxo: ATIVO → UTILIZADO / EXPIRADO / CANCELADO (apenas a partir de ATIVO)
Cancelamento	Ao cancelar um voucher ativo, os pontos utilizados retornam ao saldo do usuário
CO2 por modelo	Cada modelo de transporte possui taxa de CO2 evitado por uso registrada
Equivalência em árvores	1 árvore absorve 22 kg de CO2 por ano (equivalência = CO2 evitado ÷ 22)

2 DESCRIÇÃO DAS FUNCIONALIDADES

2.1 Principais Funcionalidades

As funcionalidades implementadas operam em duas frentes: regras de negócio na camada de serviço e persistência dos dados no banco Oracle, ambas acessadas por endpoints HTTP.

- **Gestão de Vouchers:** conversão de pontos em vouchers (aplicando bônus para perfis Premium), validação antifraude, utilização do voucher pelo titular, alteração de status (cancelar/expirar), consulta por código ou por usuário e exclusão.
- **Cálculo de Sustentabilidade:** ao utilizar um voucher, o impacto do usuário é atualizado automaticamente (CO2 evitado, viagens, equivalência em árvores e pontos investidos). O registro também pode ser consultado, criado, atualizado e removido diretamente em */impactos*.
- **CRUD de Usuários:** criar, consultar, atualizar e excluir usuários comuns e Premium, além de adicionar pontos ao saldo.
- **CRUD de Transportes:** criar, consultar, atualizar e excluir transportes públicos e transportes integrados (com sistema de bilhetagem).

2.2 Arquitetura em Camadas

O projeto segue a separação de responsabilidades apresentada em sala. Cada camada depende apenas da camada imediatamente abaixo, e a dependência da persistência é feita por interfaces (princípio da inversão de dependência).

Quadro 2 — Camadas e pacotes do projeto

Pacote	Papel	Conteúdo
controller	Resource (camada de entrada HTTP)	UsuarioController, TransporteController, VoucherController, ImpactoAmbientalController, RespostaDeErro
service	BO — regras de negócio e validações	UsuarioService, TransporteService, VoucherService, ImpactoAmbientalService
dominio	Entidades e contratos de persistência	Usuario, UsuarioPremium, TransportePublico, TransporteIntegrado, Voucher, StatusVoucher, ImpactoAmbiental e interfaces Repositorio*
infra.dao	Acesso ao banco Oracle via JDBC	UsuarioDAO, TransporteDAO, VoucherDAO, ImpactoAmbientalDAO, ConnectionFactory, ErroSql
dto	Objetos de entrada de pedidos específicos	ConversaoRequest, UtilizarRequest, StatusRequest, PontosRequest
util / config	Apoio e configuração	Validador, Numeros, CorsFilter

2.3 Endpoints da API

A API é servida por Spring Boot e responde em JSON. Os quadros a seguir listam as URIs, os verbos HTTP e os códigos de status retornados por cada recurso. Os parâmetros entre chaves são variáveis de caminho (*path variables*).

Quadro 3 — Endpoints do recurso /usuarios

Verbo	URI	Descrição	Sucesso	Erros
GET	/usuarios	Lista todos os usuários	200	500
GET	/usuarios/{id}	Busca um usuário pelo id	200	404, 500
POST	/usuarios	Cadastra usuário comum	201	400, 500
POST	/usuarios/premium	Cadastra usuário Premium	201	400, 500
PUT	/usuarios/{id}	Atualiza usuário comum	200	400, 404, 500
PUT	/usuarios/premium/{id}	Atualiza usuário Premium	200	400, 404, 500
DELETE	/usuarios/{id}	Exclui o usuário	204	404, 500
POST	/usuarios/{id}/pontos	Adiciona pontos ao saldo	200	400, 404, 500

Quadro 4 — Endpoints do recurso /transportes

Verbo	URI	Descrição	Sucesso	Erros
GET	/transportes	Lista todos os transportes	200	500
GET	/transportes/{id}	Busca um transporte pelo id	200	404, 500

Verbo	URI	Descrição	Sucesso	Erros
POST	/transportes	Cadastra transporte público	201	400, 500
POST	/transportes/integrados	Cadastra transporte integrado	201	400, 500
PUT	/transportes/{id}	Atualiza transporte público	200	400, 404, 500
PUT	/transportes/integrados/{id}	Atualiza transporte integrado	200	400, 404, 500
DELETE	/transportes/{id}	Exclui o transporte	204	404, 500

Quadro 5 — Endpoints do recurso /vouchers

Verbo	URI	Descrição	Sucesso	Erros
GET	/vouchers	Lista todos os vouchers	200	500
GET	/vouchers/usuario/ {usuarioId}	Lista os vouchers de um usuário	200	400, 500
GET	/vouchers/{codigo}	Busca um voucher pelo código	200	404, 500
POST	/vouchers	Converte pontos em voucher (usuarioId, transporteId, percentualCobertura)	201	400, 500
PUT	/vouchers/{codigo}/utilizar	Utiliza o voucher (usuarioId) e registra o impacto	200	400, 404, 500
PUT	/vouchers/{codigo}	Altera o status para CANCELADO ou EXPIRADO	200	400, 404, 500
DELETE	/vouchers/{codigo}	Exclui o voucher	204	404, 500

Quadro 6 — Endpoints do recurso /impactos

Verbo	URI	Descrição	Sucesso	Erros
GET	/impactos	Lista o impacto de todos os usuários	200	500
GET	/impactos/{usuarioId}	Impacto ambiental de um usuário	200	404, 500
POST	/impactos	Cria o registro de impacto de um usuário	201	400, 500
PUT	/impactos/{usuarioId}	Atualiza o registro de impacto	200	400, 404, 500
DELETE	/impactos/{usuarioId}	Exclui o registro de impacto	204	404, 500

Exemplo de corpo (JSON) — POST /usuarios e POST /vouchers

```
POST /usuarios
{ "id": "U001", "nome": "Ana Souza", "email": "ana@email.com",
  "cpf": "52998224725" }

POST /vouchers
{ "usuarioId": "U001", "transporteId": "T001", "percentualCobertura": 50 }
```

2.4 Tratamento de Erros e Códigos de Status

Cada controller captura as exceções lançadas pela camada de serviço e devolve uma resposta padronizada por meio da classe *RespostaDeErro*. As validações de negócio e de formato lançam *IllegalArgumentException*; já falhas inesperadas (por exemplo, erro de banco) retornam erro interno.

Quadro 7 — Códigos de status HTTP utilizados

Código	Significado	Quando ocorre
200 OK	Requisição atendida	Consultas e atualizações bem-sucedidas
201 Created	Recurso criado	POST de usuários, transportes, vouchers e impactos
204 No Content	Exclusão concluída	DELETE bem-sucedido
400 Bad Request	Dados inválidos ou regra de negócio violada	Campos obrigatórios ausentes, saldo insuficiente, limite diário atingido, voucher de outro usuário
404 Not Found	Recurso inexistente	Id, código ou usuário não encontrado
500 Internal Server Error	Falha inesperada	Erro de acesso ao banco de dados

3 PROTÓTIPO DO SISTEMA

As Figuras 1a a 1d apresentam as telas implementadas do front-end web do Urbanlink, publicado em https://urbanlink-beryl.vercel.app e integrado à API por requisições HTTP. As telas mostram a página inicial, o catálogo de transportes, a consulta dos vouchers de um usuário e o detalhe de um voucher emitido.

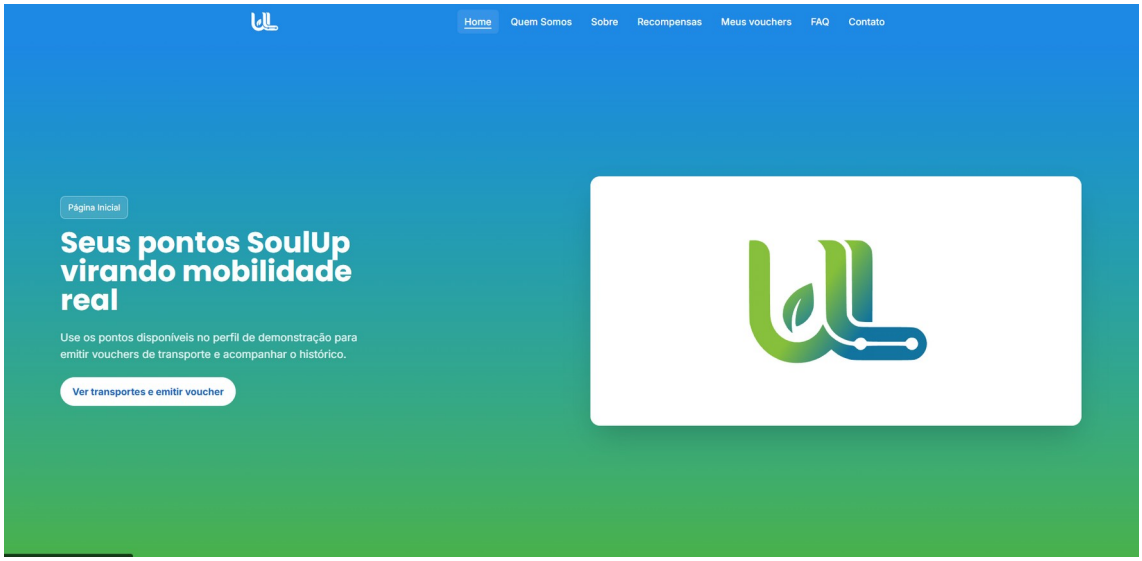


Figura 1a — Página inicial do sistema web

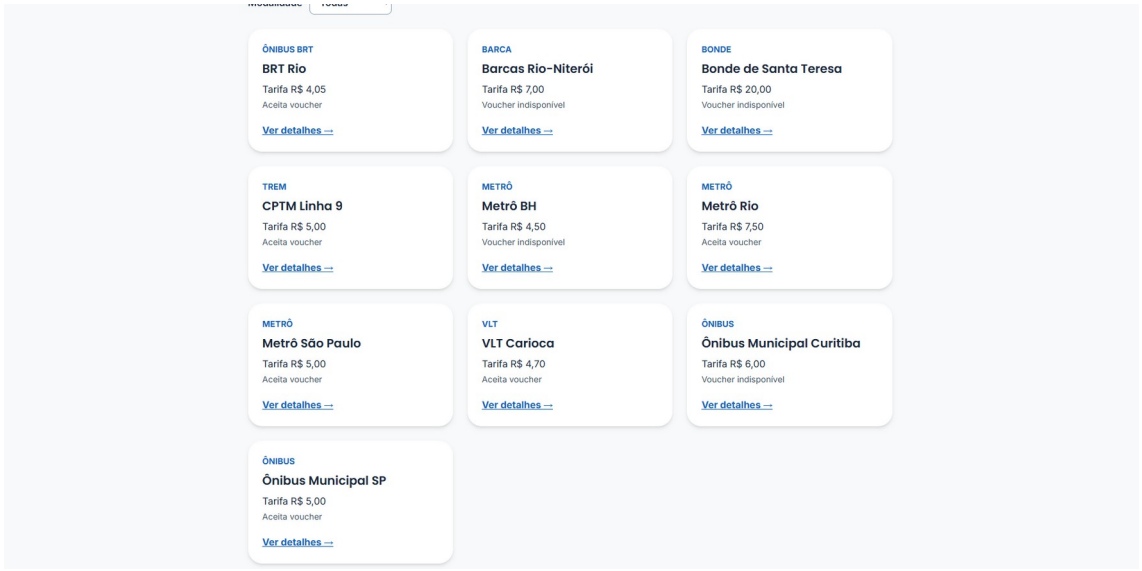


Figura 1b — Catálogo de transportes, com tarifa e disponibilidade de voucher

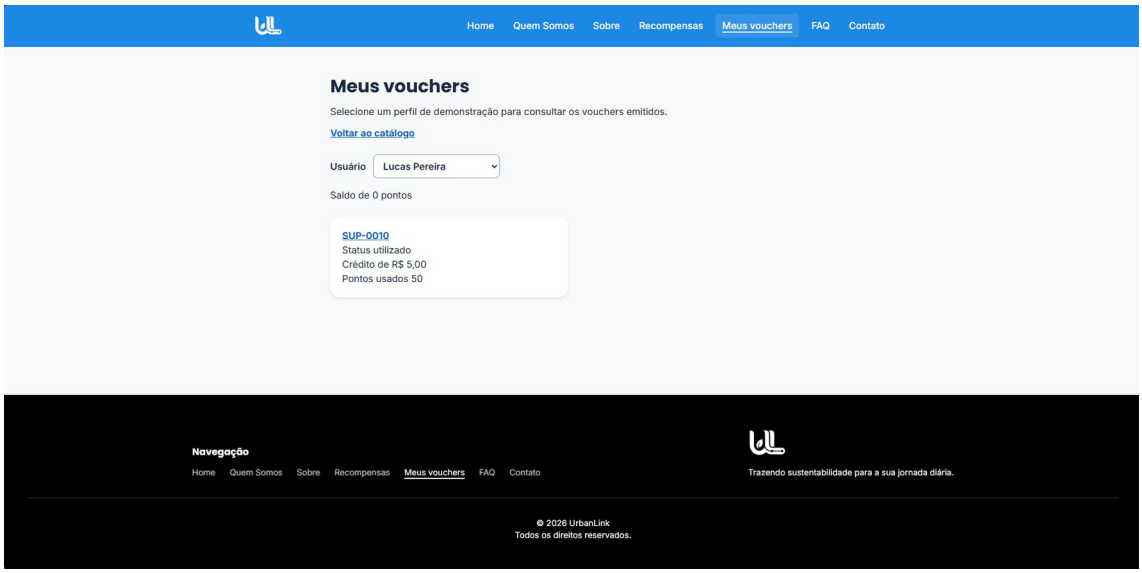


Figura 1c — Tela Meus vouchers: saldo de pontos e vouchers emitidos

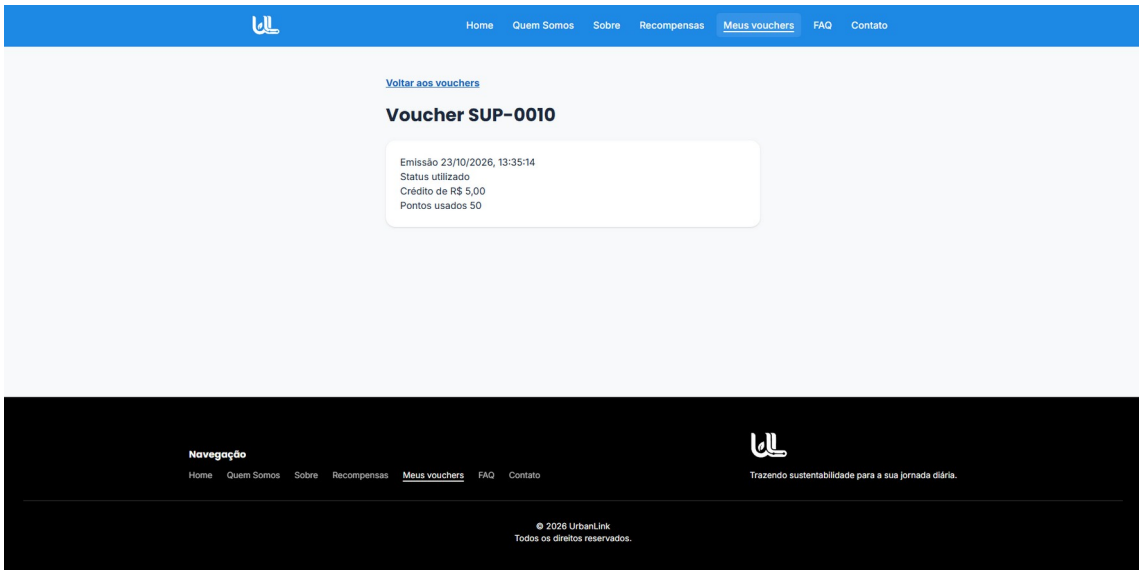


Figura 1d — Detalhe de um voucher emitido

3.1 Descrição das Telas

Quadro 8 — Telas implementadas e serviços da API relacionados

Tela	Descrição	Endpoint relacionado
Home	Página inicial com a apresentação do módulo e o botão para ver os transportes e emitir voucher	Sem chamada à API (página de apresentação)
Catálogo de transportes	Lista de transportes com modalidade, tarifa e indicação se aceita voucher, com filtro por modalidade	GET /transportes, GET /transportes/{id}
Meus vouchers	Seleção de um usuário, saldo de pontos e lista dos vouchers emitidos, com status, crédito e pontos usados	GET /usuarios, GET /usuarios/{id}, GET /vouchers/usuario/{usuarioId}
Detalhe do voucher	Dados de um voucher: data de emissão, status, crédito em reais e pontos usados	GET /vouchers/{codigo}

4 MODELO DE ENTIDADE-RELACIONAMENTO (MER)

A Figura 2 apresenta o modelo físico do banco de dados Oracle, com as entidades TB_USUARIO, TB_TRANSPORTE, TB_VOUCHER e TB_IMPACTO_AMBIENTAL, seus atributos, chaves e relacionamentos, conforme o script *schema.sql* do projeto. As heranças do modelo de classes foram mapeadas em tabelas únicas: usuários Premium são identificados pela coluna *tipo* e transportes integrados pela coluna *integrado*.

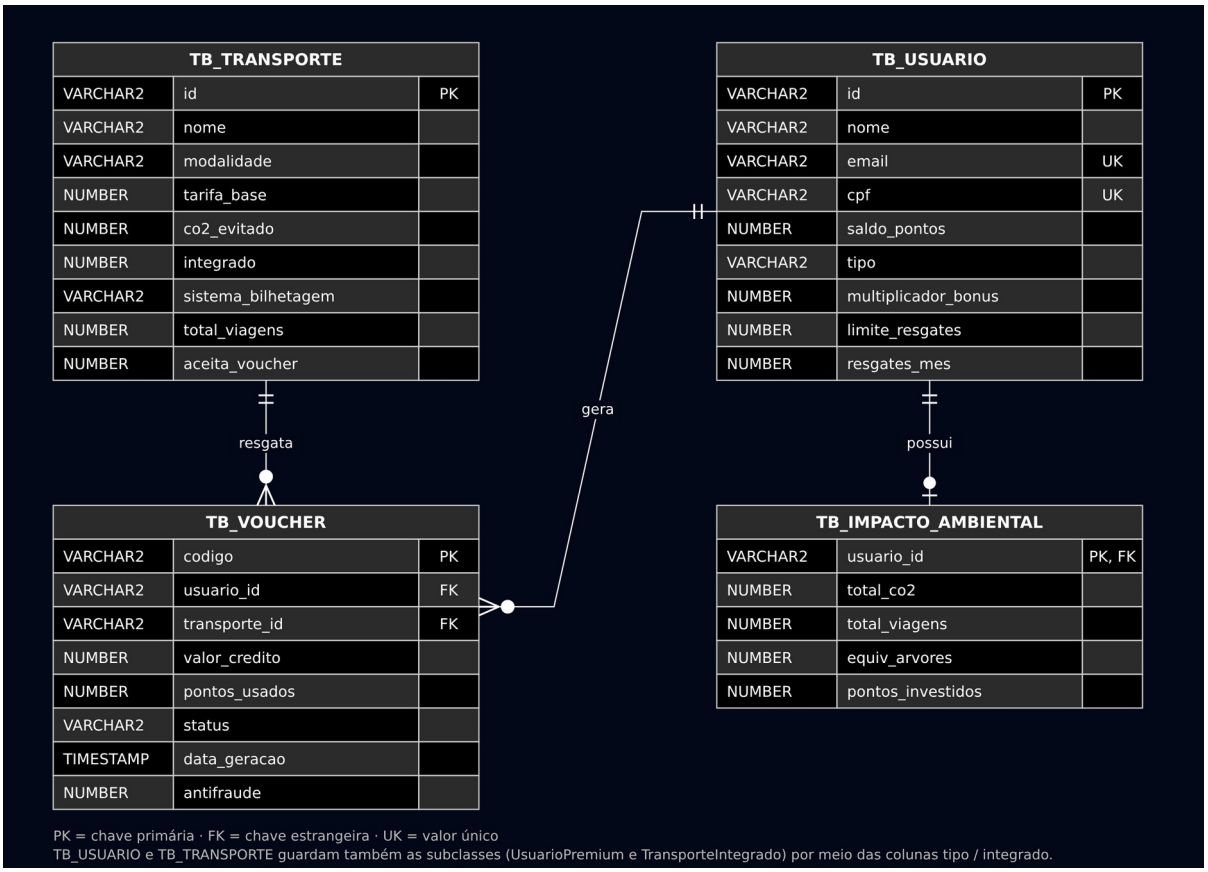


Figura 2 — Modelo de Entidade-Relacionamento

4.1 Entidades e Relacionamentos

Quadro 9 — Relacionamentos do MER

Relacionamento	Cardinalidade	Descrição
TB_USUARIO gera TB_VOUCHER	1 : N	Um usuário pode gerar vários vouchers; cada voucher pertence a um único usuário (FK usuario_id)
TB_TRANSPORTE resgata TB_VOUCHER	1 : N	Um transporte pode ter vários vouchers emitidos para ele (FK transporte_id)
TB_USUARIO possui TB_IMPACTO_AMBIENTAL	1 : 1	Cada usuário possui no máximo um registro de impacto; a chave primária é a própria FK usuario_id, com exclusão em cascata

5 DIAGRAMA DE CLASSES

5.1 Visão Geral das Classes

O sistema é composto por pacotes divididos em modelo de domínio, serviços (regras de negócio), acesso a dados (DAO) e controladores REST.

Quadro 10 — Classes e responsabilidades

Classe	Responsabilidade
Usuario	Entidade base do sistema com atributos de identificação e saldo de pontos
UsuarioPremium	Herda Usuario e adiciona multiplicador de bônus e controle de resgates mensais
TransportePublico	Representa um modelo de transporte com tarifa base e taxa de CO2 evitado; calcula pontos e valor do crédito
TransporteIntegrado	Herda TransportePublico e adiciona integração com sistema de bilhetagem
Voucher	Crédito de transporte gerado na conversão; contém código único, status e marca antifraude
StatusVoucher	Enumeração com o ciclo de vida do voucher (ATIVO, UTILIZADO, EXPIRADO, CANCELADO)
ImpactoAmbiental	Registra e calcula o CO2 evitado e a equivalência em árvores por usuário
*Controller (4 classes)	Recebem as requisições HTTP, chamam o service e montam a resposta com o código de status
*Service (4 classes)	Aplicam as regras de negócio (conversão, antifraude, validações) e orquestram os repositórios
Repositorio* (4 interfaces)	Contratos de persistência usados pelos services (inversão de dependência)
*DAO (4 classes)	Implementam os repositórios com JDBC e executam o CRUD no Oracle
ConnectionFactory	Estabelece a conexão centralizada com o banco de dados Oracle, lendo as credenciais de variáveis de ambiente

5.2 Representação UML

A Figura 3 mostra as classes de domínio, com atributos, métodos e heranças. A Figura 4 (página seguinte) mostra a organização em camadas e as dependências entre controllers, services, interfaces de repositório, DAOs e a ConnectionFactory.

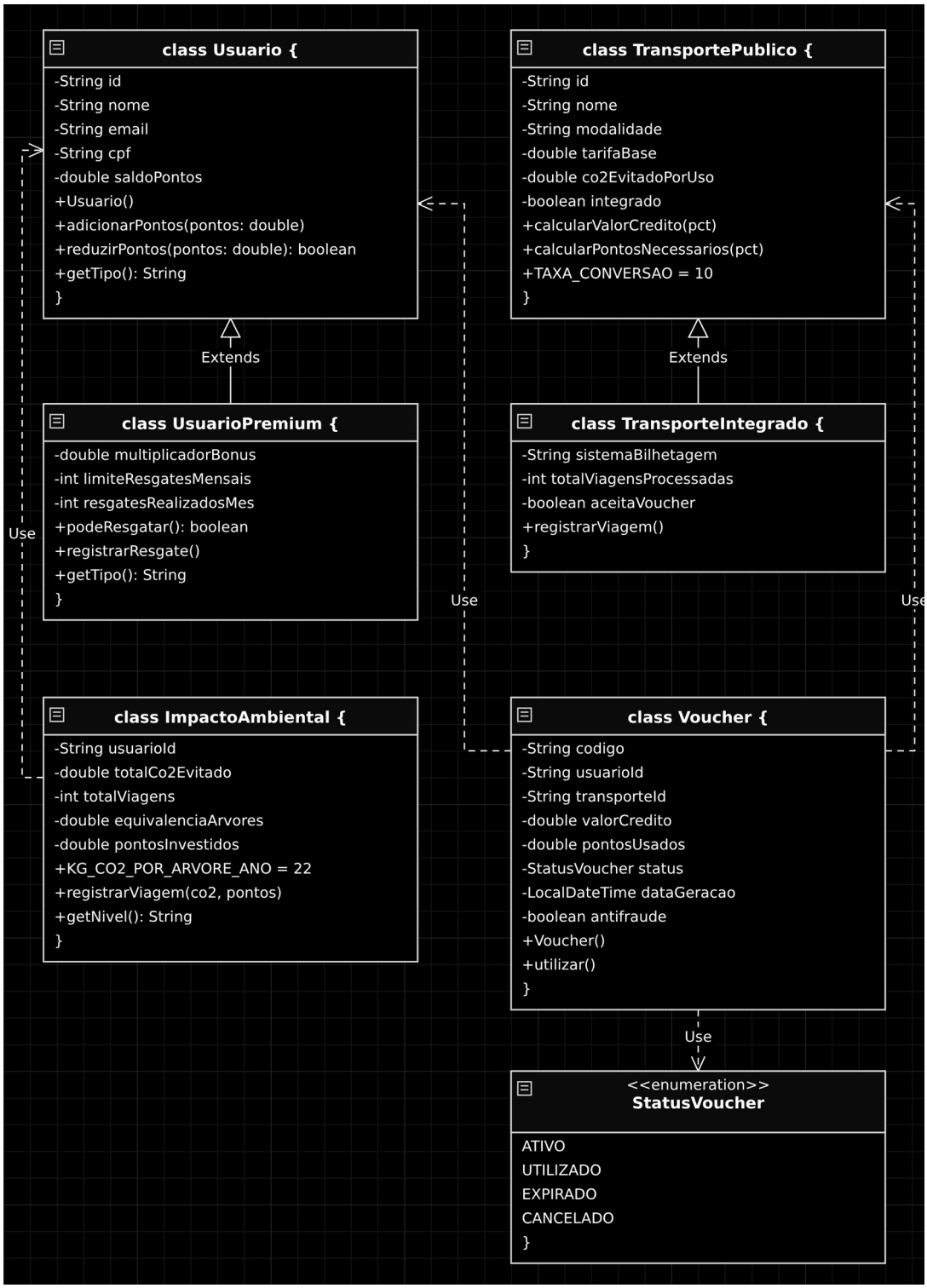


Figura 3 — Diagrama de classes UML: modelo de domínio

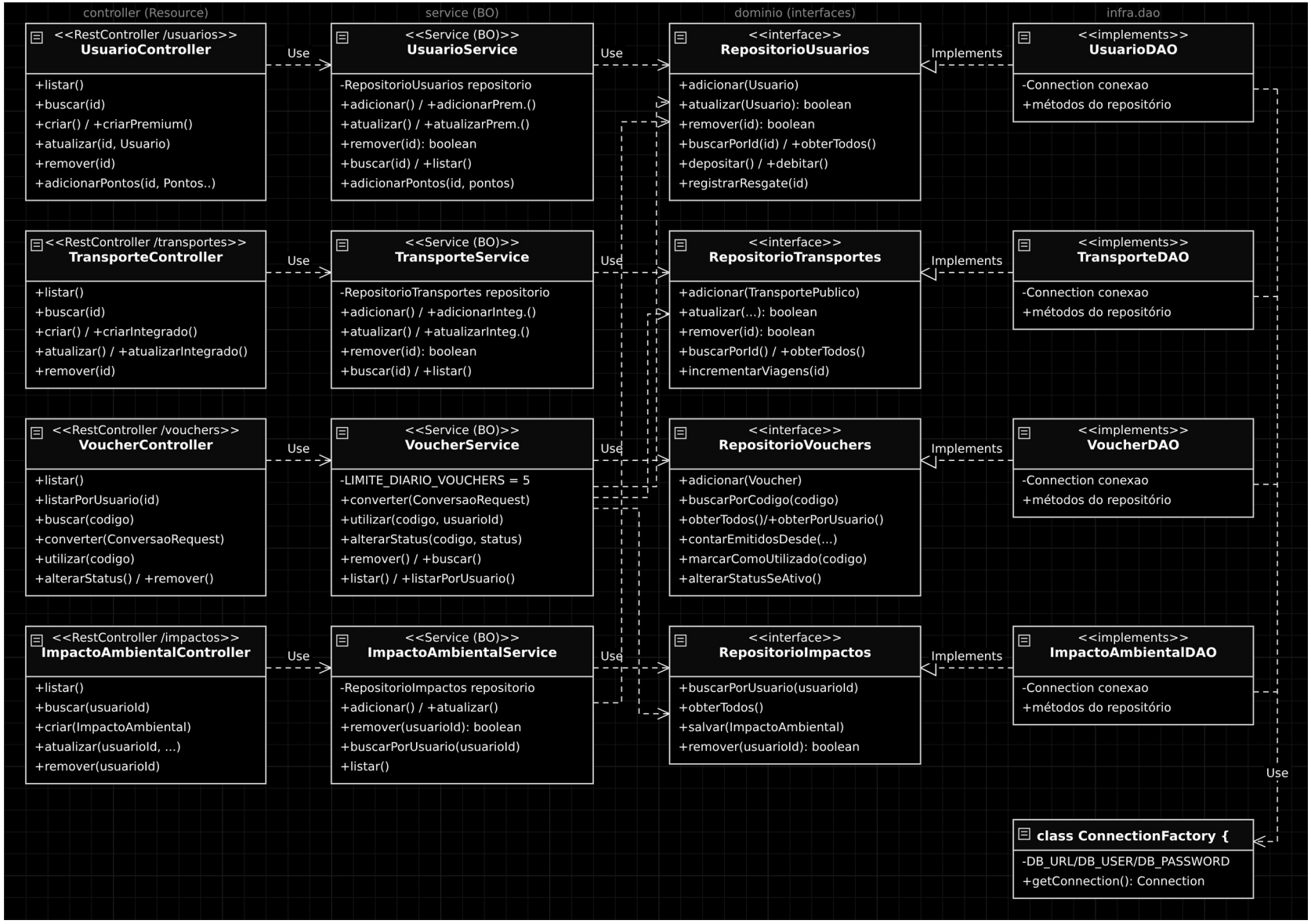


Figura 4 — Diagrama de classes UML: camadas controller, service, dominio e infra.dao

5.3 Descrição dos Relacionamentos

- **Herança:** UsuarioPremium herda de Usuario. TransporteIntegrado herda de TransportePublico. O polimorfismo é usado no VoucherService (usuario instanceof UsuarioPremium) para aplicar o multiplicador de bônus.
- **Associação entre entidades:** Voucher referencia Usuario (usuarioId) e TransportePublico (transporteId); ImpactoAmbiental referencia Usuario (usuarioId); Voucher possui um StatusVoucher.
- **Dependências entre camadas:** cada Controller utiliza o seu Service; cada Service utiliza interfaces Repositorio*. O VoucherService também utiliza os repositórios de usuários, transportes e impactos, e o ImpactoAmbientalService utiliza o repositório de usuários.
- **Realização de interface:** UsuarioDAO, TransporteDAO, VoucherDAO e ImpactoAmbientalDAO implementam, respectivamente, RepositorioUsuarios, RepositorioTransportes, RepositorioVouchers e RepositorioImpactos.
- **Dependências de persistência:** todos os DAOs dependem da classe ConnectionFactory para obter acesso ao banco de dados.
- **Coleções:** as listagens são retornadas como ArrayList, e o status e os perfis são tratados por enum e herança.

6 PROCEDIMENTOS PARA RODAR A APLICAÇÃO

6.1 Ferramentas Utilizadas

- **IDE:** IntelliJ IDEA (Community ou Ultimate).
- **Framework:** Spring Boot 4.1.2 (módulo web MVC) com build Maven.
- **Driver JDBC:** Oracle JDBC (ojdbc), baixado automaticamente pelo Maven.
- **Banco de dados:** Oracle (instância oracle.fiap.com.br:1521:ORCL).
- **Testes manuais:** Postman, Insomnia ou navegador (para requisições GET).
- **Implantação:** Docker (Dockerfile na raiz) e plataforma Render.

6.2 Versões Necessárias

- **Java JDK:** 25 (versão configurada em *java.version* no pom.xml). Caso o computador utilize outra versão, ajuste a propriedade no pom.xml e a imagem do Dockerfile para a mesma versão instalada.
- **Maven:** não é necessário instalar; o Maven integrado ao IntelliJ baixa as dependências automaticamente a partir do pom.xml.

6.3 Como Importar o Projeto

1. Extraia o arquivo urbanlink-api.zip ou clone o repositório no GitHub: https://github.com/Felipecmf/Urbanlink_sboot
2. Abra o IntelliJ IDEA → File > Open → selecione a pasta urbanlink-api (a que contém o pom.xml).
3. Aguarde o IntelliJ importar o projeto Maven e baixar as dependências.
4. Confirme em File > Project Structure > Project se o SDK selecionado é o JDK compatível com o projeto.

6.4 Variáveis de Ambiente e Banco de Dados

As credenciais não ficam no código. A ConnectionFactory lê as variáveis de ambiente abaixo, que devem ser configuradas em Run > Edit Configurations > Environment variables.

Quadro 11 — Variáveis de ambiente

Variável	Descrição	Exemplo
DB_URL	URL JDBC do Oracle	jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL
DB_USER	Usuário do banco (RM)	RM000000
DB_PASSWORD	Senha do banco	(senha pessoal)
CORS_ORIGIN	Origem do front-end autorizada a chamar a API	http://localhost:5173
PORT	Porta da API (opcional, padrão 8080)	8080

Antes da primeira execução, rode o script *database/schema.sql* no Oracle (por exemplo, pelo SQL Developer ou pelo cliente de banco do IntelliJ) para criar as tabelas usuario, transporte, voucher e impacto_ambiental.

6.5 Passo a Passo de Execução

1. Configure as variáveis de ambiente descritas no quadro 11.
2. Localize a classe principal *br.com.urbanlink.UrbanlinkApplication*.
3. Clique com o botão direito no arquivo → Run 'UrbanlinkApplication' (ou use o botão ▶️ verde ao lado do método main).
4. Aguarde a mensagem de que a aplicação iniciou na porta 8080.
5. Acesse http://localhost:8080/usuarios no navegador: a resposta deve ser uma lista em JSON.
6. Para executar os testes unitários, use mvn test (no terminal, na pasta do pom.xml) ou Run 'All Tests' na pasta src/test.

6.6 Como Testar a API

Com a aplicação em execução, siga a sequência abaixo no Postman para validar o fluxo principal:

1. POST /usuarios para cadastrar um usuário e POST /usuarios/{id}/pontos para adicionar saldo.
2. POST /transportes/integrados para cadastrar um transporte que aceita voucher.
3. POST /vouchers informando usuarioId, transporteId e percentualCobertura (retorna 201 e o código do voucher).
4. PUT /vouchers/{codigo}/utilizar informando o usuarioId do titular (retorna 200).
5. GET /impactos/{usuarioId} para conferir o CO2 evitado e a equivalência em árvores.

6.7 Publicação (Deploy)

A API é publicada na plataforma Render a partir do Dockerfile do projeto, com as mesmas variáveis de ambiente do quadro 11 configuradas no painel do serviço. URL pública da API: https://urbanlink-sboot.onrender.com. O serviço utiliza o plano gratuito da plataforma, portanto, após um período sem acesso, a primeira requisição pode levar cerca de 1 minuto para ser respondida.

7 CONSIDERAÇÕES FINAIS

A versão atual do sistema apresenta as seguintes limitações conhecidas:

- As operações que alteram mais de uma tabela (por exemplo, converter pontos e emitir o voucher) não usam transação; em caso de falha, o service devolve os pontos manualmente.
- Algumas violações de regra retornam 400, quando o ideal seria 403 ou 409.
- O contador mensal de resgates dos usuários Premium não é zerado automaticamente.
- O impacto ambiental é registrado no momento em que o voucher é utilizado.
- O saldo de pontos é armazenado como número decimal (double), sem tratamento específico de arredondamento monetário.